北 京 邮 电 大 学

本科毕业设计（论文）开题报告

学院	计算机学院（国家示范性软件学院）		专业	计算机科学与技术	
学生姓名	宋健	学号	2022212702	班级	2022211305
指导教师姓名	金大海	所在单位	北京邮电大学	职称	副教授
设计（论文）题目	（中文）大模型驱动的代码质量多维度评估系统的设计与实现				
	（英文）Design and Implementation of Multidimensional Code Quality Evaluation System Driven by Large Language Models				

毕业设计（论文）开题报告内容：（主要包含选题的背景和意义；研究的基本内容和拟解决的主要问题；研究方法及措施；研究工作的步骤与进度；主要参考文献等项目。不少于 1500 字）

一、选题背景与意义

1.1 选题背景

随着大模型（Large Language Model, LLM）技术在软件工程领域的规模化应用，AI 辅助编程已成为工业级开发的主流模式。据 Shehab 等（2024）的研究数据显示，超过 40%的工业级代码开发依赖 AI 生成或辅助优化。然而，AI 生成代码在质量管控上仍面临严峻挑战，Sabra 等（2025）指出 AI 生成代码的漏洞率高达 37%，且在一致性、精简性等维度存在显著缺陷。

当前代码质量评估存在两类核心模式，但均存在明显短板：一是**传统自动评估模式**，以 SonarQube 为代表的工具依赖静态规则匹配，仅能覆盖语法错误、简单漏洞等基础指标，无法满足 ISO/IEC 25010:2023 国际标准中“功能适用性、性能效率、兼容性”等 9 大维度的完整评估需求，更缺乏对完整工程项目的跨文件依赖分析能力（香佳宏等，2025）；二是**纯 LLM 评估模式**，虽展现出语义理解优势，但存在关键缺陷：Pathak 等（2025）发现其缺乏专用评分标准（Rubric）导致评估结果主观性强；Wang 等（2025）的实证研究表明其对复杂质量维度的覆盖不全；且现有方案多为单体架构，未支持模型、评估标准、指标的灵活配置，也缺乏中间结果的追溯与可解释性（Tong 等，2024）。

在此背景下，构建一款完全遵循 ISO/IEC 25010:2023 标准、支持可配置化与微服务化、兼具高效性与可解释性的大模型驱动多维度代码质量评估系统，成为解决当前代码质量管控痛点的关键。

1.2 选题意义

（1）理论意义

- 突破传统“规则驱动”与“纯 LLM 驱动”的局限，建立“LLM+静态工具增强+ISO 全维度锚定”的混合评估理论体系，弥补现有研究在标准化、全维度覆盖方面的空白。
- 基于 Memory RAG 与知识图谱技术，解决 LLM 超长上下文处理难题，为大模型在大规模代码评估中的工程化应用提供理论支撑（Hou 等，2024）。

- 构建可解释、可追溯的自动化评估框架，通过 Agent 的 MCP（Multi-Capability Extension）能力扩展评估边界，丰富大模型在软件工程领域的应用方法论（Xia 等，2022）。

（2）实践意义

- 实现 **ISO 全维度覆盖**：严格遵循 ISO/IEC 25010:2023 标准，覆盖 9 大核心质量维度，解决传统工具维度覆盖不全的问题（ISO/IEC 25010:2023）。
- 提升评估效率与灵活性：通过可配置化设计（模型、标准、指标）适配不同场景，支持 20 万行以上代码的批量评估，大幅降低人工审查成本（Sun 等，2025）。
- 强化工程化实用性：微服务架构支持分布式部署与灵活扩展，生成包含“问题定位-推理依据-优化建议”的结构化报告，确保评估结果可追溯、可落地（Wong 等，2025）。

二、研究现状

现有代码质量评估研究围绕“传统自动评估-LLM 驱动评估”两个阶段演进，但均未形成兼顾 ISO 全维度、工程化、可解释性的解决方案，具体现状如下：

2.1 传统自动评估技术研究

- 以静态分析工具为核心，如 SonarQube 通过预设规则检测代码重复率、语法错误等指标，但仅能覆盖 ISO 标准中 30%的质量维度，缺乏对功能适用性、兼容性、交互能力等语义层面的评估（姜佳君等，2021）。
- 动态测试工具如 EvoSuite 通过生成测试用例验证正确性，但存在测试覆盖率不足（平均 60%以下）、无法适配复杂业务逻辑的问题（Fraser 等，2011）。
- 架构层面多为单体设计，不支持评估标准与指标的灵活配置，难以适配不同项目的个性化需求（Hou 等，2024）。

2.2 LLM 驱动评估研究

- 评估方法创新：Pathak 等（2025）提出“Rubric 驱动的 LLM 评估”方法，通过专用评分标准提升一致性；Tong 等（2024）开发的 CodeJudge 框架实现了多维度评估，但未覆盖 ISO 标准中的兼容性、交互能力等维度。
- 技术瓶颈：纯 LLM 评估存在三大缺陷：一是对 ISO 标准的适配不完整，仅聚焦正确性、可读性等少数维度；二是超长上下文处理能力不足，无法支持完整工程级评估；三是评估结果不可解释，缺乏中间推理过程与证据链（香佳宏等，2025）。

2.3 标准化与工程化研究

ISO/IEC 25010:2023 标准明确了 9 大核心质量维度及细分指标，但现有评估工具均未完全覆盖，且缺乏将标准指标转化为可计算、自动化评估逻辑的成熟方案（ISO/IEC 25010:2023）；工程化方面，现有系统多未采用微服务架构，扩展性差，且未实现模型、指标、评估标准的可配置化（Sun 等，2025）。

三、研究的基本内容和拟解决的主要问题

3.1 研究基本内容

（1）大模型选型与增强方案设计

- 集成 Memory RAG 技术，将代码依赖关系、中间评估结果存入知识库，解决超长上下文处理问题；引入轻量级代码结构图谱（Code Structure Graph），构建“文件-类-函数”关联网络，提升跨文件依赖评估的准确性（Zeng 等，2022）。
- 为 Agent 配置 MCP 能力，集成静态分析工具（Tree-sitter）、性能测试工具（Locust）、漏洞检测工具（CodeQL），扩展评估工具链覆盖范围。

（2）ISO 全维度评估指标体系构建（基于 ISO/IEC 25010:2023）

严格遵循 ISO 标准，构建 9 大核心维度+细分指标的评估体系，每个指标明确自动化计算思路：

1. 功能适用性
    - 完整性：满足的功能点数量/项目特性列表（README）功能点数量（通过 LLM 提取 README 特性与代码实现匹配）。
    - 正确性：自动生成单测的通过率（基于 Pynguin/EvoSuite 生成测试用例，通过率≥配置阈值（默认 90%）为合格）（Fraser 等，2011）。
    - 适当性：代码功能与预期场景的匹配度（LLM 计算代码注释/文档与代码语义相似度，≥配置阈值（默认 85%）为合格）。
2. 性能效率
    - 时间行为：接口平均响应时间/基准版本响应时间（通过 Locust 模拟并发请求测试，比值≤配置阈值（默认 1.2）为合格）。
    - 资源利用率：内存峰值占用/基准版本占用（通过 Docker 监控，无显著异常增长为合格）。
    - 容量：系统稳定支持的最大并发用户数/设计阈值（通过压力测试，≥配置阈值（默认 90%）为合格）。
3. 兼容性
    - 共存性：代码在多环境（Windows/Linux/macOS）的编译通过率（≥配置阈值（默认 95%）为合格）。
    - 互操作性：与依赖组件的接口调用成功率（通过 Mock 测试，≥配置阈值（默认 99%）为合格）。
4. 交互能力
    - 可学习性：API 文档完整性+命名规范符合度（文档覆盖率≥配置阈值（默认 90%）且命名规范符合率≥配置阈值（默认 85%）为合格）。
    - 可操作性：接口调用复杂度（平均调用步骤≤配置阈值（默认 3 步）为合格）。
    - 包容性：代码对特殊输入（空值、异常格式）的处理能力（无崩溃且返回合理提示率≥配置阈值（默认 90%）为合格）。
5. 可靠性
    - 无故障性：连续运行 24 小时的故障次数（≤配置阈值（默认 1 次）为合格）。
    - 可用性：系统正常运行时间占比（≥配置阈值（默认 99.9%）为合格）。
    - 容错性：异常场景下的系统恢复时间（≤配置阈值（默认 5 秒）为合格）。
6. 安全性

- 保密性：敏感数据（密码、密钥）硬编码检测通过率（通过 CodeQL 检测，通过率≥配置阈值（默认 100%）为合格）。
- 完整性：关键数据操作的校验逻辑覆盖率（≥配置阈值（默认 90%）为合格）。
- 抗性：常见漏洞模式（CWE Top 25）的扫描通过率（通过 CodeQL 静态扫描，无高危漏洞为合格）。

7. 可维护性
   - 模块性：无循环依赖的模块占比（≥配置阈值（默认 90%）为合格，通过依赖分析工具检测）。
   - 可分析性：代码注释覆盖率（≥配置阈值（默认 30%）为合格）。
   - 可测试性：单元测试覆盖率（≥配置阈值（默认 70%）为合格）。
8. 灵活性
   - 适应性：代码适配不同业务场景的修改量（平均修改行数≤配置阈值（默认 10 行）为合格）。
   - 可扩展性：新增功能时的代码复用率（≥配置阈值（默认 60%）为合格）。
   - 可替换性：核心模块的接口标准化程度（≥配置阈值（默认 90%）为合格）。
9. 安全（**Safety**）
   - 风险识别：高风险代码片段占比（≤配置阈值（默认 5%）为合格，通过 LLM 结合安全规则检测）。
   - 故障安全：故障时数据丢失率（≤配置阈值（默认 0）为合格）。
   - 安全集成：与安全组件的兼容适配度（无冲突且功能正常为合格）。

（3）可配置化微服务架构设计

- 微服务拆分（基于 Python FastAPI）：拆分为模型管理服务、指标配置服务、代码解析服务、评估执行服务、报告生成服务、结果追溯服务、数据存储服务。
- 可配置化实现：支持模型类型（GPT-4/CodeLlama 等）、评估标准（ISO 标准/企业自定义）、指标权重的动态配置（通过配置中心实现）（Sun 等，2025）。
- 中间结果追溯：记录每个评估维度的推理过程、数据来源、工具输出日志，支持问题溯源与全链路查询。

（4）评估流程优化与自动化验证

- 专用提示词工程：为 9 大维度及细分指标分别设计 Rubric 驱动的提示词，强制模型输出“问题定位-推理依据-评分结果”结构化数据（Pathak 等，2025）。
- 评估流程设计：代码解析→上下文提取→多工具协同评估→LLM 整合结果→结构化报告生成。
- 大规模验证：构建包含 50 个开源项目（涵盖 Java、Python、C++）的测试数据集（累计代码量≥20 万行），与 SonarQube 等主流工具对比，验证系统准确率与维度覆盖度。

3.2 拟解决的主要问题

1. 传统评估工具 ISO 维度覆盖不全、纯 LLM 评估主观偏差大的问题，通过 ISO 全维度锚定+混合评估架构实现精准评估。
2. 超长上下文处理与跨文件依赖分析难题，通过 Memory RAG+知识图谱技术突破限制。
3. 评估结果不可解释、不可追溯的问题，通过结构化报告与中间过程日志实现全链路可查。
4. 系统灵活性与扩展性不足的问题，通过微服务化与可配置化设计适配不同应用场景。

四、研究方法及措施

4.1 阶段一：前期准备

1. 文献与标准调研：系统研读 ISO/IEC 25010:2023 标准、10 篇以上核心参考文献，梳理评估指标与技术路线。
2. 数据集构建：收集 50 个开源项目（Java/Python/C++各不少于 15 个），累计代码量≥20 万行，整理项目 README/文档、引入 Defects4J/QuixBugs 等包含已知缺陷的数据集作为评估基准。
3. 工具选型与环境搭建：确定开发工具（Vue3、FastAPI、Tree-sitter、Locust）、模型接口（OpenAI API/CodeLlama 开源部署）、数据库（PostgreSQL、Redis），搭建本地开发与测试环境。

4.2 阶段二：核心技术预研

1. 大模型选型测试：设计测试用例（涵盖 9 大维度典型场景），对比 CodeLlama、GPT-4 Code、Qwen-Coder 的评估准确率与推理速度，筛选最优模型。
2. Memory RAG 与知识图谱实现：
    - 基于 LangChain 构建 Memory RAG 模块，实现代码片段、依赖关系的存储与快速查询。
    - 使用 Neo4j 构建代码结构图谱，定义“文件-类-函数”关联关系，支持跨文件依赖分析。
3. Agent MCP 能力集成：开发工具调用接口，实现 LLM 对 Tree-sitter（代码解析）、Locust（性能测试）、CodeQL（漏洞检测）的自动调用与结果整合。

4.3 阶段三：系统设计

1. 微服务架构设计：绘制系统架构图、模块流程图、接口文档，明确各服务职责与交互逻辑。
2. 可配置化方案设计：设计模型配置表、指标权重配置表、评估标准配置表，实现动态调整功能。
3. 评估流程与提示词设计：
    - 设计“代码解析→多工具评估→LLM 整合→报告生成”的自动化流程。
    - 为 9 大维度及细分指标设计专用提示词，确保输出结构化数据。
4. 数据库设计：设计评估任务表、中间结果表、报告表、日志表，支持数据追溯与查询。

4.4 阶段四：系统开发

1. 后端服务开发：
    - 模型管理服务：实现模型注册、调用、负载均衡功能。
    - 指标配置服务：支持 ISO 指标的启用/禁用、权重调整。
    - 代码解析服务：集成 Tree-sitter 与知识图谱，实现代码结构与依赖关系提取。
    - 评估执行服务：集成 Agent MCP 能力，实现多工具协同评估与 LLM 结果整合。
    - 报告生成服务：生成可视化报告（雷达图、缺陷分布热力图）与结构化文档。
    - 结果追溯服务：记录评估全流程日志，支持中间结果查询。
2. 前端开发：实现项目上传、配置管理、评估任务提交、报告查看、结果追溯等功能。

3. 集成测试：解决服务间接口调用问题，确保系统流程顺畅运行。

4.5 阶段五：系统测试与优化

1. 功能测试：针对 9 大维度设计测试用例，验证每个指标的计算准确性与评估流程完整性。
2. 性能测试：通过 20 万行代码批量评估，测试系统响应时间（≤30 分钟）、并发处理能力（支持 5 个并行任务）。
3. 准确率优化：与 SonarQube 等工具对比评估结果，调整提示词与指标计算逻辑，将评估准确率提升至 90%以上。
4. 兼容性测试：验证系统在不同操作系统（Windows/Linux/macOS）、浏览器（Chrome/Firefox/Edge）的运行稳定性。

4.6 阶段六：系统验证与论文撰写

1. 大规模数据集验证：使用构建的 50 个开源项目数据集进行评估，统计系统的维度覆盖度、准确率、效率指标。
2. 论文撰写：整理实验数据、系统设计文档、测试报告，撰写毕业论文，完成摘要、引言、文献综述、系统设计、实验验证等章节。
3. 答辩准备：制作答辩 PPT，梳理系统创新点、设计方案与实验成果，准备演示流程。

五、研究工作的步骤与进度

周次	核心任务	具体产出
第 1-2 周	文献与 ISO 标准调研	文献综述报告、ISO 指标拆解文档
第 3-4 周	数据集构建与环境搭建	50 个开源项目数据集、开发环境部署完成
第 5-6 周	大模型选型测试	模型对比测试报告、最优模型确定
第 7-8 周	Memory RAG 与知识图谱实现	RAG 模块代码、知识图谱构建脚本
第 9-10 周	微服务架构与可配置化设计	架构图、接口文档、配置表设计
第 11-12 周	评估流程与提示词设计	流程设计文档、9 大维度专用提示词库
第 13-14 周	模型管理服务+指标配置服务开发	服务代码、单元测试用例
第 15-16 周	代码解析服务+知识图谱集成	服务代码、代码依赖分析功能实现
第 17-18 周	评估执行服务+Agent MCP 集成	服务代码、工具自动调用功能实现
第 19-20 周	报告生成服务+结果追溯服务开发	服务代码、报告生成与日志查询功能
第 21-22 周	前端开发	前端页面、前后端联调完成

第 23-24 周	系统集成测试	集成测试报告、Bug 修复完成
第 25-26 周	功能测试与准确率优化	功能测试报告、准确率优化记录
第 27-28 周	性能测试与兼容性测试	性能测试报告、兼容性测试报告
第 29-30 周	大规模数据集验证	验证报告、对比分析文档
第 31-32 周	论文撰写与答辩准备	毕业论文初稿、答辩 PPT

六、预期成果

1. **原型系统一套**：大模型驱动的代码质量多维度评估系统（微服务架构），支持 ISO/IEC 25010:2023 全维度评估，可配置化模型、指标、评估标准，评估准确率≥90%。
2. **毕业论文一篇**：字数约 2.5 万字，系统阐述 ISO 指标体系构建、核心技术实现、系统设计与实验验证结果。
3. **实验验证报告一份**：包含 50 个开源项目的测试数据、与主流工具的对比结果、系统性能指标（响应时间、并发能力）。
4. **专用资源库**：ISO 指标计算手册、专用提示词库、测试数据集（含评估基准）、系统源码与部署文档。

七、主要参考文献

1. **ISO/IEC 25010:2023**. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models.
2. **Pathak, A., et al.** (2025). "Rubric Is All You Need: Improving LLM-Based Code Evaluation With Question-Specific Rubrics." *Proceedings of the 2025 ACM Conference on International Computing Education Research*, V.1, pp. 181–95.
3. **Wang, R., et al.** (2025). "Can LLMs Replace Human Evaluators? An Empirical Study of LLM-as-a-Judge in Software Engineering." *Proceedings of the ACM on Software Engineering*, vol. 2, no. ISSTA, pp. 1955–77.
4. **Sabra, A., et al.** (2025). "Assessing the Quality and Security of AI-Generated Code: A Quantitative Analysis." *arXiv preprint arXiv:2508.14727*.
5. **Tong, W., Zhang, T., et al.** (2024). "CodeJudge: Evaluating Code Generation with Large Language Models." *Accepted to EMNLP 2024*. arXiv:2410.02184.
6. **Sun, T., Xu, J., et al.** (2025). "BitsAI-CR: Automated Code Review via LLM in Practice." *arXiv preprint arXiv:2501.15134*.
7. **Wong, S., et al.** (2025). "A Note on Code Quality Score: LLMs for Maintainable Large Codebases." *arXiv preprint arXiv:2508.02732*.
8. **Shehab, M. A., et al.** (2024). "Evaluating Large Language Models for Code Generation: Assessing Accuracy, Quality, and Performance." *2024 2nd International Conference on Foundation and Large Language Models (FLLM)*, pp. 407-416.
9. **香佳宏, 徐霄阳, 等.** (2025). "大模型在软件缺陷检测与修复的应用发展综述." *软件学报*, 36(4): 1489-1529.

10. **赵衔麟, 潘兴禄, 等.** (2025). "面向代码注释生成任务的注释质量评价研究." *软件学报*, 36(8): 3744-3768.
11. **陈梓豪, 金大海, 宫云战.** (2025). "基于漏洞异构图的图卷积网络漏洞检测方法." *小型微型计算机系统*, 46(03): 697-703.
12. **Chen, M., Tworek, J., et al.** (2021). "Evaluating large language models trained on code." *arXiv preprint arXiv:2107.03374*.
13. **Touvron, H., Lavril, T., et al.** (2023). "LLaMA: Open and efficient foundation language models." *arXiv preprint arXiv:2302.13971*.
14. **Hou, X. Y., Zhao, Y. J., et al.** (2024). "Large language models for software engineering: A systematic literature review." *arXiv preprint arXiv:2308.10620*.
15. **Huang, K., Meng, X. Y., et al.** (2023). "An empirical study on fine-tuning large language models of code for automated program repair." *Proc. of the 38th IEEE/ACM Int'l Conf. on Automated Software Engineering (ASE)*, pp. 1162–1174.
16. **Kang, S., Yoon, J., Yoo, S.** (2023). "Large language models are few-shot testers: Exploring LLM-based general bug reproduction." *Proc. of the 45th Int'l Conf. on Software Engineering (ICSE)*, pp. 2312–2323.
17. **Deng, Y. L., Xia, C. S., et al.** (2023). "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models." *Proc. of the 32nd ACM SIGSOFT Int'l Symp. on Software Testing and Analysis*, pp. 423–435.
18. **Wang, J. J., Huang, Y. C., et al.** (2024). "Software testing with large language models: Survey, landscape, and vision." *arXiv preprint arXiv:2307.07221*.
19. **OpenAI.** (2024). "GPT-4 technical report." *arXiv preprint arXiv:2303.08774*.
20. **姜佳君, 陈俊洁, 熊英飞.** (2021). "软件缺陷自动修复技术综述." *软件学报*, 32(9): 2665–2690.
21. **Xia, C. S., Wei, Y. X., Zhang, L. M.** (2022). "Practical program repair in the era of large pre-trained language models." *arXiv preprint arXiv:2210.14179*.
22. **Chowdhery, A., Narang, S., et al.** (2022). "PaLM: Scaling language modeling with pathways." *arXiv preprint arXiv:2204.02311*.
23. **Zeng, Z. R., Tan, H. Z., et al.** (2022). "An extensive study on pre-trained models for program understanding and generation." *Proc. of the 31st ACM SIGSOFT Int'l Symp. on Software Testing and Analysis*, pp. 39–51.
24. **Fraser, G., & Arcuri, A.** (2011). "EvoSuite: automatic test suite generation for object-oriented software." *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, pp. 416-419.

允许进入毕业设计（论文）下一阶段：是 ☑ 否 ☐		指导教师	
日期	2025 年 12 月 26 日	签字	

注：可根据开题报告的长度加页